ชื่อ-นามสกุล _____ รหัสนักศึกษา _____

เฉลยชุดฝึกกลางภาคและเกณฑ์ตรวจ

CPE334 Software Engineering ภาคการศึกษาที่ 1/2569

วิธีใช้: อ่านคำตอบตัวอย่างก่อน แล้วใช้ **เกณฑ์ตรวจ** ตรวจว่าตอบครบหรือไม่ ไม่ต้องคัดเกณฑ์ลงคำตอบ ข้อ 1 ใช้รูป “เป็น... เพราะ...” พร้อมวิธีตรวจและสิ่งที่ต้องยืนยัน ข้อ 2.1 แยกครบ 8 องค์ประกอบ ส่วน 2.2 เขียนเป็นย่อหน้าเดียวได้ รับคำตอบอื่นที่ตรงโจทย์และมีเหตุผล ไม่หักความผิดต้นเหตุเดียวซ้ำ

แยกที่มา: ป้าย “คำอธิบายเสริม” ใช้กับข้อเสนอหรือตัวอย่างที่เติมเอง ไม่ใช่ข้อมูลที่ผู้สอนยืนยัน เลขหน้าอ้างอิงนับจากหน้าแรกของ PDF; ชื่อไฟล์เต็มอยู่ท้ายเฉลย

ส่วน	ข้อ	คะแนน
1 ความต้องการและความปลอดภัย	1-4	25
2 การวางแผนและดำเนินโครงการ	5-7	25
3 UML/UI และ Git	8-12	50
	รวม	100

คะแนนย่อยและเกณฑ์ตรวจเป็นของชุดฝึกนี้ ไม่ใช่ประกาศหรือเฉลยทางการของผู้สอน

ข้อ 1 วิเคราะห์ความต้องการ (8 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 5-6; Lecture 2 หน้า 14-26

คำย่อ: FR (Functional Requirement) บอกหน้าที่; NFR (Non-functional Requirement) บอกระดับคุณภาพ

1.1 ดู workshop ที่เปิดลงทะเบียน 2 คะแนน

เป็น FR เพราะให้นักศึกษาที่เข้าสู่ระบบดู workshop เพื่อเลือกสมัคร; คงข้อความได้หากตกลงคำว่า “เปิดรับ” แล้ว มิฉะนั้นต้องถามเพิ่ม; ทดสอบรายการเปิด/ปิด ต้องเห็นรายการเปิดครบและไม่เห็นรายการปิด

1.2 ลงทะเบียนรวดเร็ว 2 คะแนน

เป็น NFR ด้าน performance เพราะเน้นความเร็ว แต่ยังวัดไม่ได้; เสนอให้คำขออย่างน้อย 95% ตอบภายใน 2 วินาทีเมื่อมีผู้ใช้พร้อมกัน 100 คน (รอยืนยัน); ใช้ load test วัดตั้งแต่ส่งคำขอถึงได้ผล ต้องผ่านเกณฑ์นี้

1.3 ไม่เกิน 3 workshop ต่อภาค 2 คะแนน

เป็น business/domain rule เพราะจำกัดสิทธิ์ตามนโยบาย; คงเกณฑ์ 3 รายการต่อภาค แต่ยืนยันสถานะที่นับ; ทดสอบมี 2 เพิ่มเป็น 3 ต้องผ่าน ส่วนมี 3 ขอรายการที่ 4 ต้องถูกปฏิเสธและจำนวนยังเป็น 3

1.4 ยกเลิกและดูรายชื่อ 2 คะแนน

เป็น FR แต่ไม่ atomic เพราะรวมสองหน้าที่; แยกเป็นนักศึกษายกเลิกการสมัคร กับผู้ประสานงานดูรายชื่อ; ยืนยันสิทธิ์และเงื่อนไขแล้วทดสอบแต่ละบทบาท ต้องยกเลิกได้ตามสิทธิ์และรายชื่อตรงข้อมูล

คำอธิบายเสริม: ข้อ 1.2 เป็นตัวอย่างค่าที่เสนอ ไม่ใช่ค่าจากโจทย์ ต้องตกลงชุดข้อมูล เครือข่าย จุดวัดและกรณีผิดพลาดก่อนใช้รับงาน; ข้อ 1.3 ต้องถามว่าการยกเลิกนับรวมโควตาหรือไม่

เกณฑ์ตรวจต่อข้อย่อย 2 คะแนน: ประเภทพร้อมเหตุผล 0.25; จุดประสงค์ 0.25; คุณภาพ 0.50; วิธีตรวจและผลคาดหวัง 0.50; ปรับหรือคงข้อความพร้อมเหตุผล 0.50 ข้อ 1.2 รับ NFR ด้าน usability หากนิยามและวิธีวัดสอดคล้องกัน

ข้อ 2 ประเมินข้อเสนอโครงการ WorkshopHub (7 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 7-8; Lecture 2 หน้า 28-29

2.1 องค์ประกอบของข้อเสนอเดิม 4 คะแนน

องค์ประกอบ	คำตอบตัวอย่าง
Problem	มี: ใช้ฟอร์มแยกและรวบรวมเอง ทำให้สมัครซ้ำและยืนยันช้า
Users	มี: นักศึกษาและผู้ประสานงาน
Solution	มี: เว็บไซต์ให้ค้นหา สมัคร และจัดการรายชื่อ
Expected value	มีบางส่วน: คาดว่าจะลดข้อผิดพลาดและงานจัดการ แต่ยังไม่มีค่าตั้งต้นหรือเป้าวัดผล
Scope	มี: รุ่นแรกเฉพาะ workshop ฟรี; การชำระเงินและใบรับรองทำภายหลัง
Risk	ไม่มี: ยังไม่ระบุความเสี่ยงและวิธีรับมือ
Effort	ยังไม่มีปริมาณแรงงาน: “ก่อนภาคหน้า” เป็นกำหนดส่ง ไม่ใช่ประมาณการคน-วัน
Evidence of completion	ไม่มี: ยังไม่มีเกณฑ์หรือหลักฐานตรวจรับ

เกณฑ์ตรวจ: องค์ประกอบละ 0.50 ให้ตามสถานะและเหตุผลที่อิงข้อความ รวม 4 คะแนน

2.2 ข้อเสนอฉบับปรับปรุง 3 คะแนน

WorkshopHub จะรวมการสมัคร workshop ฟรีไว้ในเว็บไซต์เดียว แทนฟอร์มแยกที่ต้องรวบรวมเอง ให้นักศึกษาค้นหาและสมัครได้ ผู้ประสานงานจัดการรายชื่อได้ เพื่อลดการสมัครซ้ำและงานจัดการ รุ่นแรกไม่รวมการชำระเงินหรือใบรับรอง และตั้งเป้าเสร็จก่อนภาคหน้า **เสนอให้ยืนยันเพิ่ม:** วัดค่าตั้งต้นแล้วตกลงเป้าลดข้อผิดพลาดและเวลาทำงาน ตรวจการสมัครซ้ำบน server และทดสอบคำขอพร้อมกัน ให้ทีมแยกงานแล้วประเมินแรงงานเป็นคน-วันก่อนยืนยันกำหนดส่ง ส่วนการรับงานให้ใช้การสาธิต ผลทดสอบที่โยงเกณฑ์ยอมรับ ผลเทียบเป้าหมาย และผู้ประสานงานตรวจรับ

คำอธิบายเสริม: ข้อมูลยังไม่พอคำนวณ effort จริง จึงไม่กำหนดตัวเลขขึ้นเอง ข้อเสนอเรื่องตัวชี้วัด ความเสี่ยง วิธีประเมินแรงงานและหลักฐานรับงานยังต้องยืนยัน ไม่ใช่ข้อตกลงเดิม

เกณฑ์ตรวจ: คงปัญหา ผู้ใช้ วิธีแก้ คุณค่าและขอบเขตเดิม 1; เติมความเสี่ยงและแนวประเมิน effort โดยระบุสิ่งที่ยังต้องยืนยัน 1; หลักฐานรับงานตรวจได้ 1 รับประมาณการเชิงตัวเลขที่ระบุสมมติฐานและมีเหตุผลรองรับ แต่ไม่บังคับตัวเลขใด

ข้อ 3 สิทธิ์การใช้งานตามบทบาท (RBAC) (4 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 9; Lecture 5 หน้า 8, 19-23, 34

RBAC (Role-Based Access Control) คือกำหนดสิทธิ์ตามบทบาท; Student = นักศึกษา, Coordinator = ผู้ประสานงาน

การกระทำ	Student	Coordinator
ดู workshop ที่เผยแพร่	Allow	Allow
ส่งคำขอลงทะเบียนของตน	Allow	Deny
สร้าง workshop	Deny	Allow
แก้ไข workshop	Deny	Allow
ส่งออกรายชื่อ	Deny	Allow

3.2: ไม่พอ เพราะผู้ใช้เรียก API โดยไม่ผ่านปุ่มได้ server ต้องยืนยันตัวตน (authentication) แล้วตรวจสิทธิ์ (authorization) จากข้อมูลที่เชื่อถือได้ ใช้ default deny: ไม่ได้รับสิทธิ์ต้องปฏิเสธ ไม่เชื่อ role ที่แนบมากับคำขอ

คำอธิบายเสริม: ตัวอย่างทดสอบที่เสนอ: ใช้บัญชี Student เรียกส่งออกรายชื่อโดยตรง แม้ปลอม role=Coordinator ต้องได้ 403 และไม่ได้รับรายชื่อ; รหัสตอบกลับนี้เป็นแบบที่เสนอ ไม่ใช่ API contract ที่โจทย์ให้

เกณฑ์ตรวจ: 3.1 ช่องละ 0.25 รวม 2.5; 3.2 แยก UI/การบังคับสิทธิ์ 0.5, ตรวจสิทธิ์ที่เชื่อถือได้และ default deny 0.5, ทดสอบตรงพร้อมผลคาดหวัง 0.5 ไม่เพิ่ม inheritance หรือกฎเจ้าของที่โจทย์ไม่ได้ให้

ข้อ 4 วิเคราะห์การอัปโหลดไฟล์ (6 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 10-11; Lecture 5 หน้า 5-8, 28-38

4.1 คำตอบตัวอย่าง (3 คะแนน)

ประเด็น	คำตอบ	คะแนน
Attacker	ผู้โจมตีคือนักศึกษาที่ข้ามการตรวจใน browser	0.5
Assets	สิ่งที่ต้องปกป้องคือรายงานผู้อื่นและ server	0.5
Attack surface	จุดโจมตีคือ API อัปโหลดและ URL ของไฟล์	0.5
Trust boundary	ข้ามขอบเขตความเชื่อถือจาก client สู่ server และจากไฟล์ข้อมูลเป็นโค้ดที่รันได้	1
Security objective	Confidentiality (การรักษาความลับ) ถูกละเมิด เพราะอ่านรายงานผู้อื่นได้	0.5

4.2 มาตรการควบคุม (control) และการทดสอบเชิงลบ (negative test) คู่ละ 1.5 คะแนน

Control	วิธีทดสอบ	ผลที่คาดหวัง
ตรวจชนิดและโครงสร้าง PDF บน server ไม่เชื่อชื่อไฟล์หรือ MIME อย่างเดียว	ส่งสคริปต์ปลอม PDF เข้า API โดยข้าม browser	ปฏิเสธไฟล์และไม่มี URL ที่ใช้รันสคริปต์นั้นได้
เก็บในพื้นที่ที่รันโค้ดไม่ได้ และตรวจสิทธิ์ก่อนดาวน์โหลด	เปิด URL ของไฟล์ทดสอบ แล้วใช้ A ขอรายงานของ B	สคริปต์ไม่ทำงาน และ A ไม่ได้ข้อมูลของ B หากไม่มีสิทธิ์

เกณฑ์ตรวจ 4.2: คู่ละมาตรการ 0.5 + วิธีทดสอบ 0.5 + ผลคาดหวัง 0.5 ตรวจเฉพาะ browser ยังไม่พอ

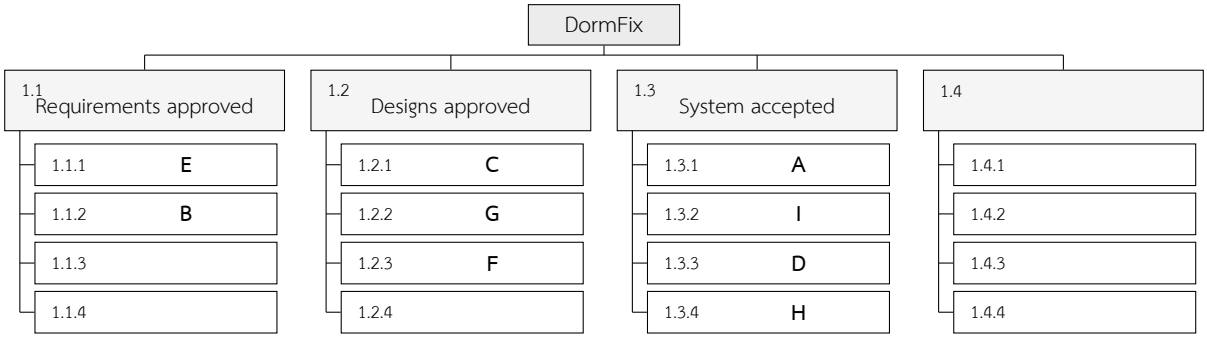
คำอธิบายเสริม: เหตุการณ์นี้เป็นโค้ดรันบน server ไม่ใช่ XSS ที่รันใน browser; integrity (ความถูกต้องของข้อมูล) และ availability (ความพร้อมใช้งาน) อาจได้รับผล แต่โจทย์ยังไม่ยืนยันว่าเสียหายแล้ว

ข้อ 5 วางแผน DormFix **(10 คะแนน)**

แหล่งตรวจ: Lecture 10 หน้า 14-21; Lecture 6 หน้า 71-96; RACI ยึดบันทึกประชุมข้อ 5.2 ของชุดฝึก

WBS (Work Breakdown Structure) ใช้แบ่งงานเป็นลำดับชั้น; milestone คือจุดสำเร็จ ไม่ใช่งานที่ใช้เวลาทำ

5.1 WBS ตามผลสำเร็จ **3 คะแนน**



เกณฑ์ตรวจ: milestone/ลำดับ 1; A-I ครบและอยู่กลุ่มถูก 1; เรียงงานภายในกลุ่มถูกและเว้นช่องไม่ใช้ 1 งาน C/G และ A/I ทำขนานกัน จึงสลับลำดับในคู่ได้ เส้น WBS แสดงลำดับชั้น ไม่ใช่ dependency

5.2 RACI **2 คะแนน**

R = Responsible (ลงมือทำ), A = Accountable (รับผิดชอบผล), C = Consulted (ให้คำปรึกษา), I = Informed (รับแจ้ง); ไม่เกี่ยวข้องใส่ “-”

งาน	PM	Aom	Bank	Chai	Dao
E	A	R	C	I	-
C	A	C	R	I	-
G	A	-	C	R	I
D	A	I	-	C	R

เกณฑ์ตรวจ: 20 ช่อง ช่องละ 0.10 ตามบันทึกประชุม รวม 2 คะแนน

5.3 กำหนดการและคำอธิบาย **5 คะแนน**

E → B → {C,G} → F → {A,I} → D → H → project buffer

เหตุผลสั้น: เผื่อ task buffer หลัง B/F/H อย่างละ 1 วัน เพราะรอคนนอกทีมที่มีงานอื่น ส่วนงานภายในใช้ project buffer 2 วันท้ายร่วมกัน; milestone ใช้เวลา 0 วันเพราะเป็นจุดบรรลุผล

งาน	ทำงาน (ม.ค. 2027)	Buffer	เริ่ม-สิ้นสุดรวม buffer
E	4, 5, 6	ไม่มี	4-6
B	7	8	7-8
C	11, 12, 13	ไม่มี	11-13
G	11, 12	ไม่มี	11-12
F	14	15	14-15
A	18, 19, 20, 21	ไม่มี	18-21
I	18, 19, 20	ไม่มี	18-20
D	22, 25	ไม่มี	22-25
H	26	27	26-27
PB	-	28, 29	28-29

ข้อ 5.3: Gantt chart ตัวอย่าง มกราคม 2027

Milestones: M1 = 8, M2 = 15, M3 = 29 ม.ค. 2027; จบวันที่ 29 รวม 20 วันทำงานพร้อม buffer Chai ทำ G ก่อน A และ Dao ทำ I ก่อน D จึงไม่ใช้คนซ้อนกัน; D เว้นเสาร์-อาทิตย์ที่ 23-24 ม.ค.

เกณฑ์ตรวจ 5.3: No./Owner 0.5; ระยะเวลา/วัน/Start-Due 1; dependency/resource 1; buffers 1; milestone 0.5; เหตุผล buffer 0.5 และ milestone ไม่เพิ่มเวลา 0.5 รวม 5 ไม่บวกคะแนนคำอธิบายซ้ำ

สัปดาห์ที่ 1-2

No.	Task	Owner	Start	Due	4	5	6	7	8	11	12	13	14	15
1.1	M1	-	8	8					*					
1.1.1	E	Aom	4	6	X	X	X							
1.1.2	B	PM	7	8				X	-					
1.2	M2	-	15	15										*
1.2.1	C	Bank	11	13						X	X	X		
1.2.2	G	Chai	11	12						X	X			
1.2.3	F	PM	14	15									X	-
														ⓐ

สัปดาห์ที่ 3-4

No.	Task	Owner	Start	Due	18	19	20	21	22	25	26	27	28	29
1.3	M3	-	29	29										*
1.3.1	A	Chai	18	21	X	X	X	X						
1.3.2	I	Dao	18	20	X	X	X							
1.3.3	D	Dao	22	25					X	X				
1.3.4	H	PM	26	27							X	-		
-	PB	-	28	29									-	-
					ⓐ									

X = วันทำงาน - = buffer * = milestone PB = project buffer

จุดเชื่อม a แสดงว่า A และ I ต้องรอ F กับ buffer ของ F ก่อนเริ่ม

M1: ข้อกำหนดอนุมัติ M2: แบบอนุมัติ M3: ตรวจรับระบบรวม project buffer

คำอธิบายเสริม: buffer และวันครบกำหนดนี้ใช้ตามเงื่อนไข DormFix ไม่ใช่กฎว่าทุกโครงการต้องเผื่อหลังงานอนุมัติทุกครั้ง

ข้อ 6 ปรับรายละเอียด Backlog (10 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 22-23; Lecture 7 หน้า 60-64, 94-104

6.1 User story: ใครต้องการอะไร เพื่ออะไร 2 คะแนน

“ในฐานะนักศึกษา ฉันต้องการจองสนามแบดมินตันครั้งละ 1 ชั่วโมง เพื่อไม่ต้องต่อคิวที่เคาน์เตอร์”

เกณฑ์ตรวจ: ผู้ใช้ 0.5; ความสามารถ 0.75; คุณค่า 0.75

6.2 รายละเอียดจากลูกค้า 2 คะแนน

มีข้อมูลสนามและเวลาเปิดแล้ว; จองครั้งละ 1 ชั่วโมง; แต่ละคนถือการจองได้ไม่เกิน 1 รายการต่อวัน; จองสำเร็จแล้วแสดง Booked ส่วนการยกเลิกและการนับสิทธิ์ วันจองล่วงหน้าและการชำระเงินยังต้องถาม

เกณฑ์ตรวจ: ข้อเท็จจริงสี่ประเด็น ประเด็นละ 0.5

6.3 เกณฑ์ยอมรับสองสถานการณ์ 4 คะแนน

สถานการณ์: จองสำเร็จ
Given นักศึกษา S ยังไม่จองวันนี้ และสนาม 2 ว่างเวลา 17:00-18:00 ซึ่งเป็นช่วงให้บริการ
When S จองสนาม 2 ช่วงนั้น
Then ระบบยืนยันการจอง 1 ชั่วโมงให้ S และแสดงช่วงนั้นเป็น Booked

สถานการณ์: เกินสิทธิ์ต่อวัน
Given S มีการจองวันนี้แล้ว 1 รายการ และมีอีกช่วงที่ว่าง
When S ขอจองเพิ่มวันนี้
Then ปฏิเสธคำขอ จำนวนการจองไม่เพิ่ม และช่วงใหม่ยังว่าง

เกณฑ์ตรวจต่อสถานการณ์: เงื่อนไข 0.5; การกระทำ 0.5; ผลและสถานะตามกฎ 1 รวมสองสถานการณ์ 4

6.4 ข้อมูลกำกับ Issue (Metadata) 2 คะแนน

Assignees: -; Labels: enhancement; Milestone: -; Estimate: -

เกณฑ์ตรวจ: ช่องละ 0.5 ไม่เดาเจ้าของ/sprint/points และไม่ใช้ 0 แทนค่ายังไม่ทราบ

ข้อ 7 วางแผน Sprint (5 คะแนน)

แหล่งตรวจ: Lecture 10 หน้า 25-26; Lecture 7 หน้า 67-72, 114-118

งาน	เรื่อง	Points	เลือก	เหตุผล
1	ดูตำแหน่งรถปัจจุบัน	8	Yes	ตรงเป้าหมายและสำคัญสูง
2	จ่ายด้วย QR	3	No	ไม่ตรงเป้าหมายรอบนี้
3	เวลาถึงป้ายของฉัน	5	Yes	ตรงเป้าหมาย
4	เตือนก่อนรถถึง	3	Yes	ช่วยเป้าหมายและใส่ได้
5	เวลาถึงทุกป้าย	5	No	ลำดับต่ำกว่าและเกินกำลังทีม
6	แก้รายการถูกตัดบนมือถือ	2	No	ไม่เหลือกำลังรับงานเพิ่ม

7.2: $8 + 5 + 3 = 16$ points **7.3:** เลือก 1/3/4 ตามเป้าหมาย Sprint Goal และลำดับความสำคัญ รวม 16 points พอดีกำลังทีม (capacity); ไม่เลือก 2 เพราะไม่ช่วยเป้าหมาย ส่วน 5/6 เกินกำลังที่เหลือ

เกณฑ์ตรวจ: points หกช่อง ช่องละ 0.25 และการเลือกหกช่อง ช่องละ 0.25 รวม 3; ผลรวม 1; เหตุผลด้านเป้าหมาย/ลำดับ 0.5 และ capacity 0.5

คำอธิบายเสริม: เทียบ anchor 3 บน scale: ใหญ่ขึ้นสองขั้นคือ 3 ไป 5 ไป 8 ไม่ใช่ 3+2; points วัดขนาดงานสัมพัทธ์ ไม่ใช่วันหรือชั่วโมง รับแผนอื่นที่มีเหตุผลจากโจทย์และไม่เกิน capacity

ข้อ 8 Sequence Diagram (10 คะแนน)

แหล่งตรวจ: Samples หน้า 1-3 (R1, R5-R11 และ scenario ข้อ 1.1); Lecture 4 หน้า 13

Sequence Diagram แสดงเหตุการณ์ตามลำดับใน scenario นี้ หมายเลขบนลูกศรใช้เทียบตาราง ไม่ใช่คะแนน เส้นทึบหัวทึบคือ call (เรียกทำงาน), เส้นประหัวเปิดคือ return (ตอบกลับ), เส้นทึบหัวเปิดคือ notification (ส่งแจ้ง)

เลข	Message หรือผลที่ต้องสื่อ
1-3	Student เข้าสู่ระบบผ่าน KUSSO; KURegistration ได้ผลยืนยันบัญชีและ role
4-5	ใช้ username ขอข้อมูลนักศึกษาจาก KUStudentInfo แล้วได้ profile กลับมา
6	เปิด Draft เดิมของภาคปัจจุบันที่มี 2 offerings พร้อม profile ไม่สร้างฟอร์มว่างทับ
7-8	เพิ่ม offering จาก List View เป็น 3 รายการ แล้วแสดงค่าลงทะเบียนรวมใหม่
9-11	ส่งฟอร์ม ตรวจทุก offering พบหนึ่ง section เต็ม จึงแจ้งนักศึกษาโดยยังไม่เริ่มชำระเงิน
12-13	นักศึกษาเปลี่ยน section ที่เต็ม ระบบปรับรายการและยอดรวม
14-16	ส่งใหม่ ตรวจที่นั่งทุก offering แล้วผ่าน จึงแสดงวิธีชำระเงิน
17-20	เลือก bank transfer ระบบเริ่มรายการกับ BBLPaymentGateway นักศึกษาโอน แล้ว gateway แจ้งผลสำเร็จ
21-24	บันทึก Done และปิดการแก้ ส่ง PDF ผ่าน KUEmail ถึงนักศึกษา แล้วแสดงผลสำเร็จ

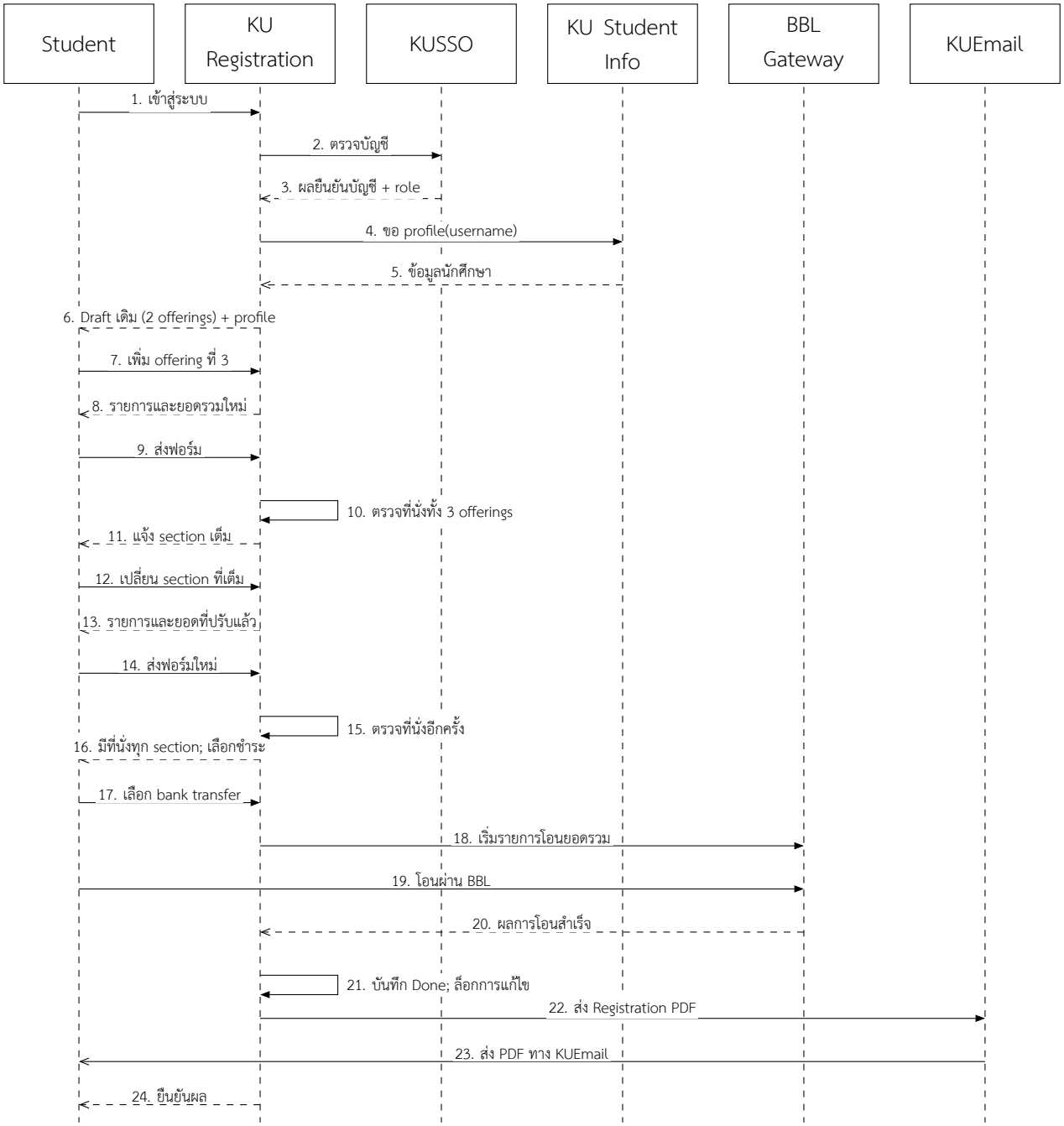
เกณฑ์ตรวจ 10 คะแนน:

1. participants/direction ถูกต้อง 1; login/profile และโหลด Draft เดิม 1 (รวม 2)
2. เพิ่ม offering และ fee รวมถูก 1; submit ตรวจ 3 offerings แล้วแจ้งเต็มโดยไม่คิดเงิน 1 (รวม 2)
3. เปลี่ยน section 1; submit และตรวจใหม่ผ่าน 1 (รวม 2)
4. bank transfer ใช้ BBL และผลสำเร็จกลับถูกทิศ 2
5. Done และ non-editable 1; PDF ส่งผ่าน KUEmail ถึง Student 1 (รวม 2)

รับการรวมข้อความหรือใช้ alt/loop หากลำดับเชิงเหตุผลยังครบ ไม่บังคับให้ตอบกลับทุกเส้นหรือใส่ activation bars เมื่อโจทย์ไม่ได้กำหนด และไม่ต้องวาด credit card เพราะ scenario นี้เลือก bank transfer

คำอธิบายเสริม: ชื่อข้อความและรายละเอียดการติดต่อระบบภายนอกในภาพเป็นแบบจำลอง ไม่ใช่ API จริงหรือข้อกำหนด protocol; อาจแสดง SSO redirect ผ่าน Student เพิ่มได้หากพฤติกรรมยังตรง R1

ข้อ 8: Sequence Diagram ตัวอย่าง



ข้อ 9 Activity Diagram **(10 คะแนน)**

แหล่งตรวจ: Samples หน้า 1-2, 4 (R1, R5-R11 และ workflow ข้อ 1.2); Lecture 4 หน้า 13, 16

คำตอบย่อ: เข้าระบบผ่าน KUSSO แล้วเปิดฟอร์มว่างพร้อมข้อมูลนักศึกษา เลือก offerings คำนวณยอดและเก็บ Draft เมื่อส่งฟอร์มให้ตรวจที่นั่งทุกรายการ หากเต็มให้เตือนและวนกลับไปแก้/ส่งใหม่ หากมีที่นั่งครบจึงเลือกบัตรผ่าน KBank หรือโอนผ่าน BBL ผลชำระไม่สำเร็จให้กลับไปเลือกวิธีชำระ; เมื่อสำเร็จให้เป็น Done ห้ามแก้ ส่ง PDF ผ่าน KUEmail และจบเมื่อนักศึกษาได้รับยืนยัน

เกณฑ์ตรวจ 10 คะแนน:

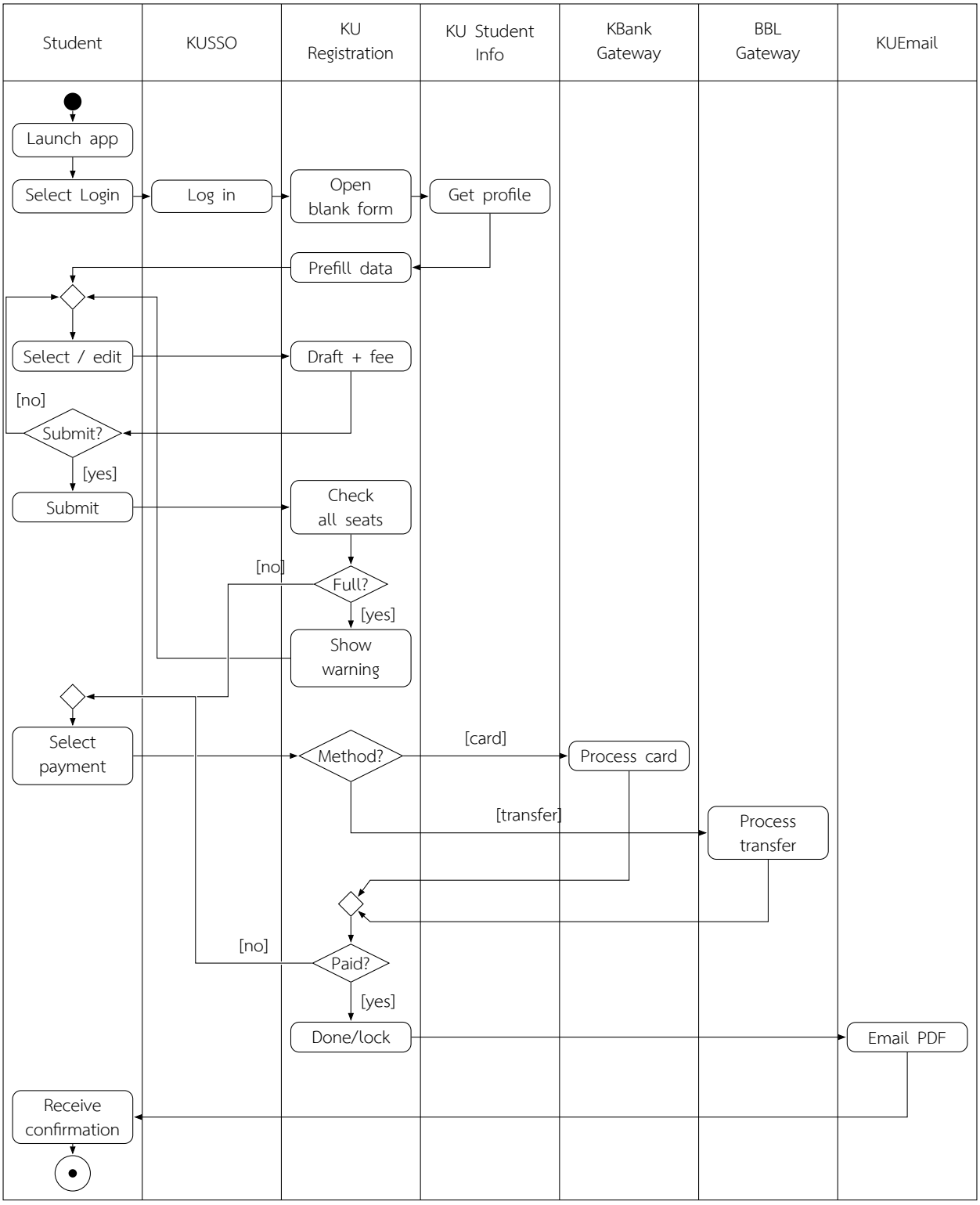
สิ่งที่ตรวจ	คะแนน
Start/login/profile/prefill พร้อม lane ถูกต้อง	1.5
เลือก/เพิ่ม offerings, fee และ Draft	1.5
ตรวจทุก offering, guards และเส้นทางเต็มย้อนกลับไปแก้/submit	2
Decision วิธีชำระมีทั้ง KBank และ BBL วาง lane ถูก รวมผลและตรวจว่าชำระสำเร็จหรือไม่	2
Done พร้อมห้ามแก้ 1; PDF email/รับยืนยัน 1; final node 0.5	2.5
Control flow อ่านตามได้ ไม่สลับเป็น sequence/lifelines	0.5
รวม	10

ภาพใช้ชื่อกิจกรรมภาษาอังกฤษตามแนว: ``Select / edit" คือเลือกหรือแก้ offering จาก List View; ``Draft + fee" คือปรับรายการและยอดรวม; ``Done/lock" คือเปลี่ยนสถานะและปิดการแก้ไขออนไลน์

รับภาพหลายหน้าที่เชื่อมต่อชัด หรือรวมกิจกรรมที่มีความหมายเดียวกันได้ ใช้ decision/merge แยกและรวมทางเลือกชำระเพียงวิธีเดียว ไม่ใช้ fork/join ที่สื่อว่าทำสองวิธีพร้อมกัน

คำอธิบายเสริม: ภาพคงทางเลือกและวงรอบตามตัวอย่างเฉลย Samples หน้า 16 รวมการชำระไม่สำเร็จแล้วลองใหม่จนสำเร็จ ไม่เพิ่มเงื่อนไขคืนเงินหรือยกเลิกรายการที่โจทย์ไม่ได้ระบุ; อย่าใช้ Draft เดิมจากข้อ 8 แทนฟอร์มว่างของข้อนี้

ข้อ 9: Activity Diagram ตัวอย่าง



ข้อ 10 List/Form UI **(10 คะแนน)**

แหล่งตรวจ: Samples หน้า 1-2, 5 (R5-R11); Lecture 4 หน้า 111-117; ข้อ 10.1 เพิ่ม List View ตามโจทย์ชุดฝึก

10.1 List View ตัวอย่าง (4 คะแนน)

Invoice ใน Samples หน้า 5 เป็นตัวอย่าง notation ให้เปลี่ยนเป็นข้อมูลและพฤติกรรมของ Registration ตาม R5-R11 ตารางหลักเชื่อมกับรายละเอียดแถวที่เลือก เพื่อแสดง R7 ครบโดยไม่อัดทุก field ในแถวเดียว

Course Offering List View Student

Course code / name: _____ Semester: active

[Search] [Clear filters]

Code	Name	Sec.	Cr.	Fee	Seats	Select
CPE201	Data Structures	2	3	1,500	5	[Add]

Selected offering details

ID: C201 | Code: CPE201 | Name: Data Structures | Credits: 3 | Fee: THB 1,500

Semester: active | Section: 2 | Room: CB401 | Class time: Tue 09:00-12:00

Display notes: Group 2 | Maximum: 40 | Registered: 35 | Seats remaining: 5

[Back to Registration Form] [Prev] 1 [Next]

คำอธิบายเสริม: ข้อมูลในหน้าจอเป็นค่าตัวอย่าง ไม่ใช่ข้อมูลจริงจากโจทย์ แถวที่เลือกต้องตรงกับส่วนรายละเอียด และใช้การ์ดที่แสดงข้อมูลครบแทนได้

เกณฑ์ตรวจ 10.1: field รวม 2 คะแนน แบ่ง 4 กลุ่ม กลุ่มละ 0.5: (id/code/name), (credits/fee), (semester/section/room/class time/notes), (maximum/registered/remaining) ให้บางส่วนภายในกลุ่มตามความครบ; controls กรองข้อมูล 0.75; เลือก offering ส่งกลับฟอร์ม 0.75; notation/readability และการเชื่อมแถวกับ detail 0.5 รวม 4 ไม่ให้คะแนนเพียงวาดกล่องชื่อ “List” โดยไม่มีข้อมูล

อ่าน notation: กรอบว่างคือ editable textbox; กรอบขีดเฉียงคือ read-only/calculated ในตารางใช้เส้นเฉียงที่หัวคอลัมน์กำกับค่าทั้งคอลัมน์ ส่วน * ใช้กับช่องบังคับที่ผู้ใช้ต้องเลือก ไม่ใช่เปิดให้แก้ profile จาก R5

10.2 Registration Form ตัวอย่าง (6 คะแนน)

Student Registration Form Status: Draft

[Close] [Save Draft] [Submit]

Student information

Student ID:	67000001	Name:	Kamon
Degree:	Computer Eng.	Email:	student@example.edu
Login name:	s67000001	Semester:	active semester

Selected offerings

*Offering		Section	Fee	Action
CPE201	[LoV]	2	1,500	[Change] [Remove]
CPE202	[LoV]	1	1,500	[Change] [Remove]

[Add from Offering List] Total fee: 3,000

When section full:
CPE201 section 2 is full. Change this offering and resubmit.

After seats OK: *Payment method ( ) Credit card / KBank ( ) Bank transfer / BBL

[Proceed to payment]

เมื่อสำเร็จ: Status เป็น Done และขีดเฉียงช่องอ่านอย่างเดียว ปิดการเพิ่ม/แก้/ลบและส่งฟอร์ม พร้อมข้อความ "Online editing is closed. Your registration PDF has been emailed."

คำอธิบายเสริม: ภาพรวมแผงเตือนเมื่อเต็มและแผงชำระหลังตรวจผ่านไว้เพื่ออธิบายทางเลือก หน้าจอจริงแสดงตามสถานะปัจจุบัน; Draft ต้องเปิดกลับมาแก้ได้ ข้อมูลตัวอย่างและ controls ที่ออกแบบเพิ่มต้องไม่กลายเป็นกฎใหม่

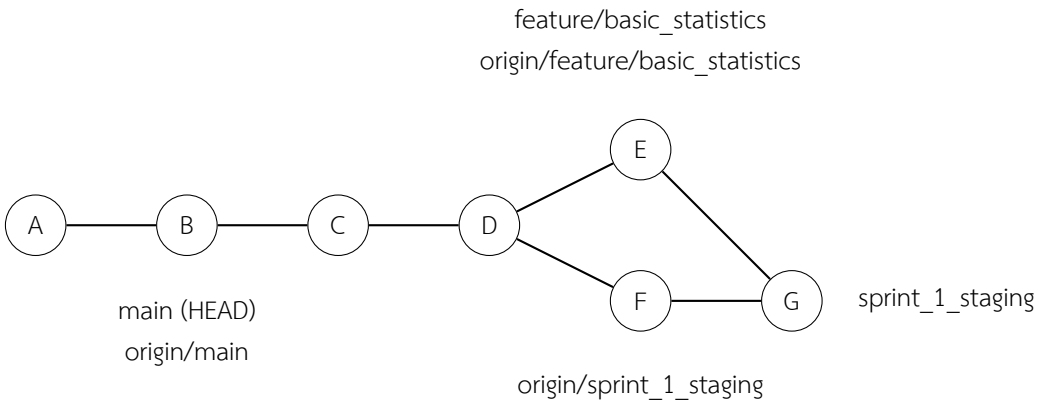
เกณฑ์ตรวจ 10.2:

- Header มี Student ID, name, degree program, email, login username จากระบบ และ active semester ชัดว่า RO: 1
- Line items เชื่อม offering/section, เพิ่มและแก้รายการได้, fee ต่อรายการกับ total คำนวณสัมพันธ์กัน: 1.5
- Draft บันทึกและกลับมาแก้; submit แล้วแสดงข้อผิดพลาดที่นั่งและแก้/resubmit ได้: 1.5
- เลือก card/KBank หรือ transfer/BBL เมื่อผ่านการตรวจ: 1
- Done ห้ามแก้ พร้อมผล PDF email: 0.5; notation required/editable/RO/calculated สอดคล้องกัน: 0.5

ไม่บังคับ registration number, registration date, total credits หรือจำนวนวิชาขั้นต่ำ เพราะ R5-R11 ไม่ได้กำหนด หากเพิ่มเพื่อช่วยใช้งานต้องไม่กลายเป็นกฎหรือภาระใหม่ที่โจทย์ไม่ได้ให้

ข้อ 11 Three-way merge แล้ว switch **(10 คะแนน)**

แหล่งตรวจ: Samples ข้อ 2.5 หน้า 12 และเฉลยหน้า 21; สมมติฐาน Git หน้า 6



Reference	Commit
main; origin/main	B
feature/basic_statistics; origin/feature/basic_statistics	E
sprint_1_staging	G
origin/sprint_1_staging	F
HEAD	main (B)

คำตอบย่อ: เป็น three-way merge เพราะสาย E และ F แยกจาก D จึงสร้าง G ที่มี parents F และ E ตามลำดับ แล้วเลื่อน sprint_1_staging ไป G ส่วน `switch main` ย้าย HEAD ไป main ที่ B และใช้ไฟล์จาก commit B โดยไม่เลื่อน branch ใดเพิ่มตัวชี้ origin/* คงเดิมเพราะไม่มีคำสั่งติดต่อ remote

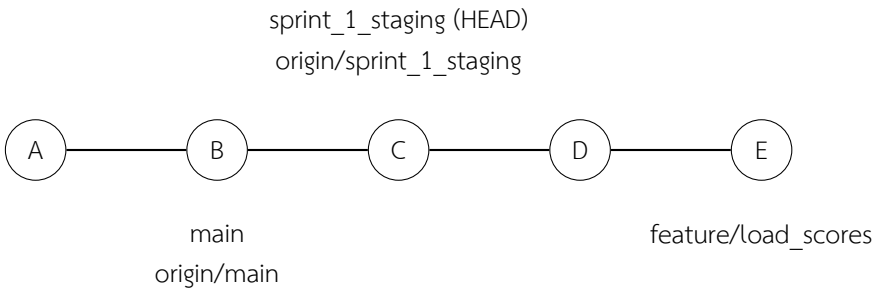
เกณฑ์ตรวจ: ancestry A-F เดิม 1; G มี parents F และ E ครบ 2; local sprint=G, feature=E, main=B ตัวละ 1 รวม 3; origin/main=B, origin/feature=E, origin/sprint=F รวม 1.5; HEAD ที่ main/B 1; three-way พร้อมเหตุผล 1; switch ไม่ย้าย branch pointers 0.5 รวม 10

ข้อ 12 คำสั่งและ graph แบบต่อเนื่อง **(10 คะแนน)**

แหล่งตรวจ: Samples ข้อ 2.3 หน้า 10-11 และเฉลยหน้า 20-21; สมมติฐาน Git หน้า 6

12.1 Switch **2 คะแนน**

```
git switch sprint_1_staging
```

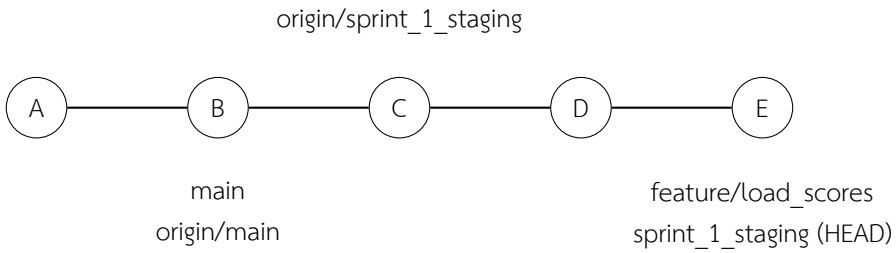


คะแนน: คำสั่ง 1; graph พร้อม HEAD ที่ C และ pointers เดิม 1

12.2 Merge **3 คะแนน**

```
git merge feature/load_scores
```

เป็น fast-forward เพราะ C เป็น ancestor ของ E ไม่สร้าง merge commit ใหม่



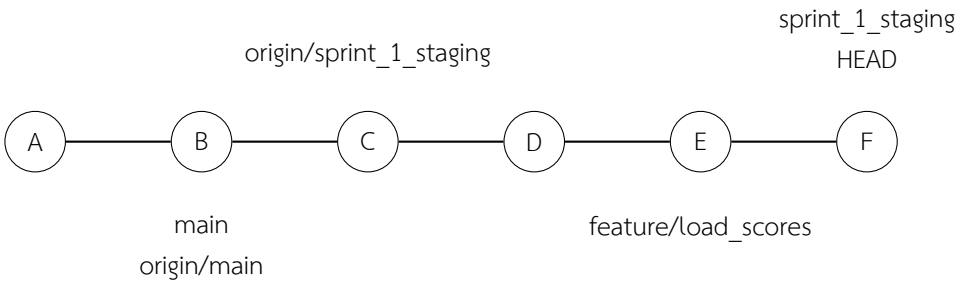
คะแนน: คำสั่ง merge ถูก branch 1; ระบุประเภท fast-forward 1; graph เลื่อน local sprint/HEAD ไป E และคง origin/sprint ที่ C 1

ข้อควรระวัง: กราฟเริ่มต้นไม่มี origin/feature/load_scores จึงไม่เติมตัวชี้นี้เอง และไม่ใช้ `git merge --no-ff` เพราะจะสร้าง merge commit ต่างจาก fast-forward ตามโจทย์

12.3 Inspect, stage และ commit **5 คะแนน**

```
git status
git diff -- score_data.py scores.csv
git add -- score_data.py scores.csv validation_notes.txt
git diff --cached -- score_data.py scores.csv validation_notes.txt
git commit -m "Document score validation cases"
```

เหตุผลสั้น: `status` ดูสถานะไฟล์; `diff` ดูส่วนต่างของไฟล์ที่ Git ติดตามแล้ว; `diff --cached` ตรวจสิ่งที่เลือกไว้หลัง stage รวมไฟล์ใหม่ ก่อนสร้าง commit รับวิธีอื่นที่ตรวจครบและเลือกเฉพาะสามไฟล์



F มี parent E ตัวเดียวและบันทึกการแก้ทั้งสามไฟล์; sprint_1_staging = F โดย HEAD ชี้ branch นี้ ส่วน feature/load_scores = E, main และ origin/main = B, origin/sprint_1_staging = C เพราะไม่มีคำสั่งติดต่อ remote จึงคง remote-tracking refs เดิม

เกณฑ์ตรวจ 5 คะแนน:

- Inspect เห็นสถานะและเนื้อหาการเปลี่ยนแปลงที่จะ commit รวมไฟล์ใหม่ 1
- Stage ชื่อไฟล์สามรายการชัดเจน ไม่ใช้ `git add .` 1
- Commit ด้วยข้อความที่กำหนด 1
- F ต่อจาก E และ local sprint/HEAD ไป F 1
- pointers อื่นคงครบและไม่สร้าง merge commit แทรก 1

หากกราฟผิดสืบเนื่องจาก 12.2 ให้ตรวจขั้นใหม่บน branch และ ancestry ที่ผู้ตอบวาดอย่างสอดคล้อง ให้คะแนนทักษะใหม่ที่ถูกต้อง ไม่หักต้นเหตุเดิมซ้ำทั้งข้อ

แหล่งอ้างอิงของเฉลย

ชื่อ Lecture/Samples ที่อ้างรายข้อหมายถึงไฟล์ด้านล่าง เลขหน้าคือลำดับหน้า PDF ไม่ใช่เลขสไลด์

Lecture 2: CPE334_Lecture02 Requirement Engineering.pdf

Lecture 4: CPE334_Lecture04 UML, SDS Features, STS Features.pdf

Lecture 5: CPE334_Lecture05 - Security in SE.pdf

Lecture 6: CPE334_Lecture06 - Project Management.pdf

Lecture 7: CPE334_Lecture07 - Project Execution.pdf

Lecture 10: CPE334_Lecture10 - Midterm Exam Prep_Group Work Project.pdf

Samples: CPE334_Midterm Samples by Suthep.pdf

เฉลยนี้ใช้ข้อความและหน้าที่อ้าง ไม่ได้ถอดทุกภาพเฉลยหรือเอกสารในลิงก์ภายนอก แผนภาพตัวอย่างวาดขึ้นสำหรับชุดฝึก ไม่ใช่ภาพเฉลยทางการ

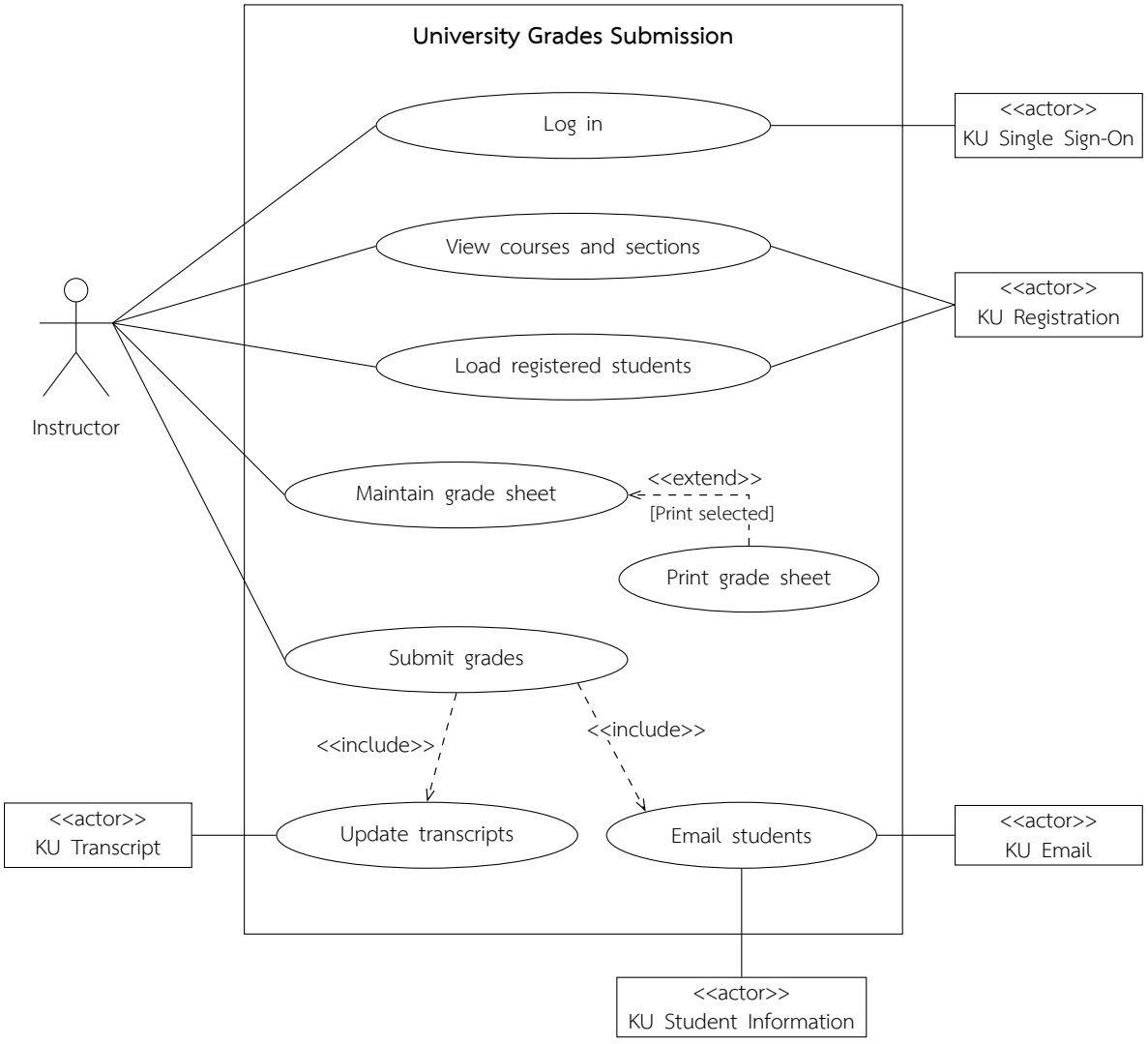
เฉลยแบบฝึกเสริม S1-S3

แบบฝึกเหล่านี้ไม่รวมใน 100 คะแนนหรือเวลา 3 ชั่วโมง ใช้รายการตรวจตนเองแทนการเพิ่มคะแนนให้ชุดหลัก

S1 Use Case: University Grades Submission

แหล่งตรวจ: Lecture 4 หน้า 5-8 (notation) และ 64-65 (requirements); ภาพประยุกต์สำหรับแบบฝึกนี้

คำตอบย่อ: Instructor เป็นผู้ใช้หลัก ระบบภายนอกทั้งห้าอยู่นอก boundary การ submit ต้องอยู่ใน Ready และต้องอัปเดต transcript/แจ้ง email ทั้งคู่ ส่วน Print เป็นทางเลือกของการดูแล grade sheet



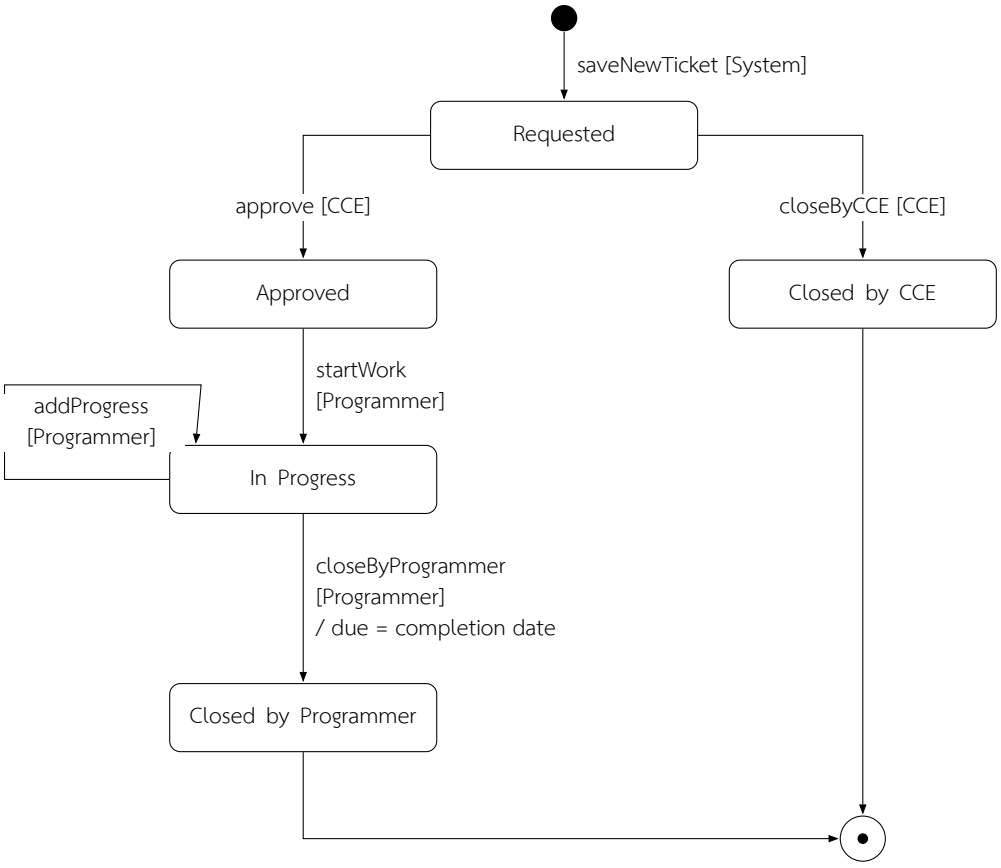
ตรวจตนเอง: actor association เป็นเส้นทึบไม่มีหัว; **<<include>>** ชี้จาก Submit ไปงานบังคับ (Update transcript/Email students); **<<extend>>** ชี้จาก Print ไป Maintain พร้อมเงื่อนไข [Print selected]. Generalization ต้องชี้จาก child ไป parent ด้วยเส้นทึบและสามเหลี่ยมโปร่งที่ parent แต่โจทย์ Grades ไม่มี actor/use case แบบ parent-child จึงไม่วาดเพิ่ม

อ่านภาพให้ถูก: ไม่ใช่ลำดับขั้นตอนการทำงาน Login เป็นเงื่อนไขก่อนใช้ฟังก์ชัน ไม่จำเป็นต้องลาก include ไป Login จากทุก use case; Student Info ให้ข้อมูลติดต่อแก่การแจ้ง email ส่วน KU Email เป็นผู้ส่ง

S2 State: Service Ticket

แบบฝึกเสริม ไม่รวม 100 คะแนน

แหล่งตรวจ: Lecture 4 หน้า 45-46 และเส้นทางภาพตัวอย่างหน้า 61-62



คำตอบย่อ: เริ่ม Requested แล้ว CCE เลือกอนุมัติหรือปิด หากอนุมัติ Programmer เริ่มเป็น In Progress เพิ่ม progress แบบ self-loop และปิดเมื่อเสร็จ ทั้งสองสถานะ Closed จบวงจรและไม่มี actions ใหม่

จุดตรวจ	สิ่งที่ต้องมี
เริ่มวงจร	save สร้าง Action แรกด้วย Requested และ Initial Due Date
บทบาท	CCE ทำเฉพาะ Requested; Programmer ใช้เส้นทาง Approved/In Progress
Self-loop	progress notes/วันนัดใหม่ไม่ทำให้หลุดจาก In Progress
ผลของ action	อัปเดต Last Status/Last Due Date; action เดิมแก้หรือลบไม่ได้
สิ้นสุด	Closed ทั้งสองแบบไป final และไม่มีเส้นทางกลับไปเพิ่ม action

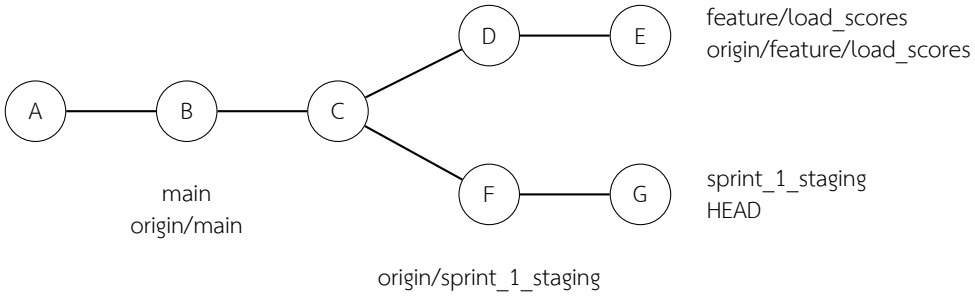
ขอบเขตภาพ: โจทย์นี้ยึดเส้นทางเริ่มงานก่อนปิดจากตัวอย่างหน้า 62 ไม่เพิ่มทางลัด Approved ไป Closed by Programmer จากการตีความข้อความหน้า 46 ที่กว้างกว่า

Notation: วงเล็บเหลี่ยมในข้อนี้ระบุ actor ตามคำสั่งหน้า 61 ไม่ใช่ Boolean guard ของ UML ทั่วไป ไม่ควรนำรูปแบบนี้ไปใช้แทน guard โดยไม่อธิบายบริบท

S3 Git: Explain the repository graph

แบบฝึกเสริม ไม่รวม 100 คะแนน

แหล่งตรวจ: Samples ข้อ 2.2 หน้า 8-9; model answer หน้า 19; ข้อสมมติหน้า 6



ข้อ	คำตอบพร้อมเหตุผล
S3.1	checkout อยู่ที่ **`sprint_1_staging`**; HEAD ชี้ branch นี้และระบุ G
S3.2	G เพราะ local sprint อยู่ที่ G แต่ **`origin/sprint_1_staging`** อยู่ที่ F ส่วน F เป็น ancestor ของ G การมี commit G ใน local จึงยังไม่แสดงว่า remote-tracking ref นี้ตามมาถึง G แล้ว
S3.3	G เพราะ working directory สะอาดและ HEAD ระบุ G ผ่าน current branch
S3.4	fast-forward ไม่ได้ เพราะ E และ G แยกสายจาก common ancestor C และไม่มีปลายใดเป็น ancestor ของอีกปลาย ต้องทำ three-way merge; หากสำเร็จจะสร้าง merge commit ที่มี parents G และ E
S3.5	พิสูจน์ไม่ได้ เพราะ graph แสดง ancestry ไม่ได้แสดงเนื้อหาบรรทัดที่แก้ การเกิด conflict ขึ้นกับการเปลี่ยนแปลงที่เข้ากันไม่ได้
S3.6	**`git fetch origin`** อาจรับ commits/objects ใหม่และอัปเดต remote-tracking refs ตาม refspec แต่ไม่ merge งาน ไม่เลื่อน local branches หรือ HEAD และไม่เปลี่ยน working directory ให้โดยอัตโนมัติ

ตัวชี้ใน snapshot ก่อนทำคำสั่งใด ๆ

Reference	Commit
main; origin/main	B
feature/load_scores; origin/feature/load_scores	E
sprint_1_staging; HEAD ผ่าน branch นี้	G
origin/sprint_1_staging	F

ตรวจตนเอง: แยก local branch ออกจาก origin/*; อ่าน parent/common ancestor ถูก; ไม่อ้างว่าแยกสายแล้วต้อง conflict; ไม่เปลี่ยนคำว่า fetch ให้หมายถึง pull/merge

ข้อจำกัดของ snapshot: origin/* บอกสถานะ remote ล่าสุดที่เครื่องรู้ ไม่ได้ยืนยัน GitHub ณ เวลาปัจจุบัน คำตอบข้อ S3.2 จึงอธิบายความต่างของ refs ที่แสดง ไม่เดาว่ามีใคร push หรือแก้ remote หลัง snapshot หรือไม่